



COP290
DESIGN PRACTICES IN PROGRAMMING

Assignment 3

Extending LevelDB: Range Scan, Range Delete & Manual Full
Compaction

Manya Jain (2024CS10351)

Prabuddha Sinha (2024CS10199)

Indian Institute of Technology Delhi

April 27, 2026

Repository Link:

https://github.com/sir12321/COP290-Assignment_3

Contents

1	Overview	3
2	Public API Changes	3
3	Execution Trace: Put	3
3.1	Entry Point - DBImpl::Put	3
3.2	Writer Queue - DBImpl::Write	4
3.3	MakeRoomForWrite	4
3.4	Batch Coalescing - BuildBatchGroup	4
3.5	WAL Append - log_>AddRecord	5
3.6	SkipList Insertion - WriteBatchInternal::InsertInto	5
3.7	Memtable Flush to SSTable	5
4	Execution Trace: Get	5
4.1	Entry Point and Snapshot Isolation - DBImpl::Get	5
4.2	Layer 1 - Active MemTable	6
4.3	Layer 2 - Immutable MemTable	6
4.4	Layer 3 - SSTables: Version::Get	6
4.5	Level-0: Linear Scan Newest-to-Oldest	6
4.6	Levels 1-6: Binary Search	6
4.7	Table Cache Lookup - TableCache::Get	7
4.8	Iterator Construction for Range Reads	7
5	Implementing Scan(start_key, end_key)	7
5.1	Design: NewIterator() vs. Internal Modification	7
5.2	Unified View via MergingIterator	7
5.3	Implementation Details	8
5.4	Interval Bounds Handling	8
5.5	Snapshot Correctness	8
6	Implementing DeleteRange(start_key, end_key)	9
6.1	Design: Discovery and Atomic Deletion	9
6.2	The TOCTOU Race Condition	9
6.3	Achieving Linearizability via an Exclusive Writer	9
6.4	Key Design Trade-Offs	11
7	Implementing ForceFullCompaction()	11
7.1	FullCompactionStats Data Structure	11
7.2	Synchronization Barrier and Concurrency Control	12
7.3	Triggering and Tracking the Compaction	12
7.4	Printing to stdout	12
7.5	Design Trade-Off: Disk I/O vs. Memory Usage	13

8	Compaction Mechanism Analysis	13
8.1	Structural Properties and Invariants	13
8.2	DoCompactionWork and File Splitting	13
8.3	Interaction with Automatic Compaction	13
9	Evaluation	14
9.1	Functional Correctness	14
9.2	Performance and Latency Considerations	14
9.3	Compaction Statistics Accuracy	14
9.4	Sample Compaction Statistics	14
9.5	Testing Methodology	15
10	Conclusion	16

1 Overview

In this report, we detail our implementation of three new APIs added to the LevelDB storage engine for COP290 Assignment 3:

API	File
Scan(start_key, end_key)	db/db_impl.cc
DeleteRange(start_key, end_key)	db/db_impl.cc
ForceFullCompaction()	db/db_impl.cc

We declared the public API additions in `include/leveldb/db.h`. Our core implementations reside within `db/db_impl.cc` and `db/db_impl.h`, while supporting read and meta-data logic natively appears in `db/version_set.cc`, `db/memtable.cc`, and `db/table_cache.cc`.

Our guiding design philosophy throughout this project was **conservative reuse**. Rather than introducing new storage formats or custom iterator types, we built every new API on top of existing LevelDB abstractions. This approach guarantees our implementation remains strictly compatible with the original Write-Ahead Log (WAL), memtable, snapshot, SSTable, and background compaction logic.

2 Public API Changes

We extended the abstract `leveldb::DB` interface in `include/leveldb/db.h` by declaring the three required pure-virtual methods:

```
virtual Status Scan(const ReadOptions& options,
                   const Slice& start_key,
                   const Slice& end_key,
                   std::vector<std::pair<std::string, std::string>>&
                   result) = 0;

virtual Status DeleteRange(const WriteOptions& write_options,
                          const Slice& start_key,
                          const Slice& end_key) = 0;

virtual Status ForceFullCompaction() = 0;
```

These signatures exactly match the assignment specifications. We provided the concrete implementations for these methods within the `DBImpl` class.

3 Execution Trace: Put

To contextualize the write mechanisms utilized in our custom extensions, we traced a standard write operation from the public API down to the physical disk structures.

3.1 Entry Point - `DBImpl::Put`

When a caller invokes `db->Put(write_options, key, value)`, `DBImpl::Put()` forwards the request to `DB::Put()`. This function constructs a `WriteBatch` on the stack, calls `batch.Put(key, value)` to serialize the key-value pair into a length-prefixed byte record, and then passes the batch to `DBImpl::Write()`.

```

Status DB::Put(const WriteOptions& opt, const Slice& key, const Slice&
    value) {
    WriteBatch batch;
    batch.Put(key, value);
    return Write(opt, &batch);
}

```

3.2 Writer Queue - DBImpl::Write

DBImpl::Write() wraps the batch in a `Writer` struct and appends it to the `writers_` deque under a mutex lock. Writers that are not at the front of the deque spin on `w.cv.Wait()` until the preceding writer completes and signals them. This single-queue discipline enforces a globally ordered sequence number for every write, preventing concurrent log or memtable corruption.

3.3 MakeRoomForWrite

Once at the front of the queue, the leading thread calls `MakeRoomForWrite(false)`. We analyzed its control loop and identified several stall conditions:

- If `bg_error_` is set, the write is immediately rejected.
- If the L0 file count reaches `kL0_SlowdownWritesTrigger` (8), writes are delayed by 1 ms to throttle throughput.
- If the active memtable retains sufficient capacity ($\text{usage} \leq \text{write_buffer_size}$, default 4 MB), the loop exits.
- If an immutable memtable is waiting to be flushed, the write stalls on `background_work_finished` until the background thread completes the flush.
- If the L0 file count reaches `kL0_StopWritesTrigger` (12), all writes stall until compaction can catch up.

If none of these stall conditions apply and the memtable is full, `MakeRoomForWrite()` rotates the tables: it promotes the active memtable to immutable, opens a new WAL file, allocates a fresh `MemTable`, and calls `MaybeScheduleCompaction()` to awaken the background thread.

3.4 Batch Coalescing - BuildBatchGroup

Before writing, the leader executes `BuildBatchGroup(&last_writer)`. This walks the `writers_` deque and coalesces compatible pending batches into a single combined batch (up to ~1 MB). We noted that writers marked with `exclusive = true` are not absorbed, immediately terminating the walk. The combined batch is then assigned a contiguous block of sequence numbers starting at `versions_->LastSequence() + 1`.

3.5 WAL Append - `log_->AddRecord`

The mutex is released, and the combined batch is appended to the Write-Ahead Log via `log_->AddRecord(WriteBatchInternal::Contents(write_batch))`. The WAL operates using 32 KB blocks, with each record featuring a 7-byte header (checksum, length, type) followed by the payload. If `WriteOptions.sync` is true, `logfile_->Sync()` executes immediately afterward to flush the OS buffer to disk, ensuring crash-recovery durability.

3.6 SkipList Insertion - `WriteBatchInternal::InsertInto`

Following a successful WAL append, `WriteBatchInternal::InsertInto(write_batch, mem_)` replays the batch into the active MemTable. Internally, this executes `MemTableInserter::Put()` and `MemTable::Add()`. We observed that `MemTable::Add()` encodes the entry as follows:

```
// Layout: [varint32 internal_key_size] [user_key bytes] [tag: seq<<8 |
           type] [varint32 val_size] [value bytes]
char* buf = arena_.Allocate(encoded_len);
//... fill buf...
table_.Insert(buf); // SkipList::Insert
```

The SkipList maintains entries in sorted internal-key order. The internal key comprises the user key and a packed 8-byte tag (`sequence_number << 8`) | `value_type`. Because higher sequence numbers sort before lower ones for identical user keys, a forward iterator naturally yields the newest version first.

3.7 Memtable Flush to SSTable

When the background thread detects `imm_ != nullptr`, it invokes `CompactMemTable()` followed by `WriteLevel0Table(imm_, &edit, base)`. This process generates a sorted SSTable on disk. We observed that the output is not strictly constrained to Level 0: `base->PickLevelForMemTableOutput()` may optimally push the file to Level 1 or 2 if it exhibits no overlap with existing files at those levels.

4 Execution Trace: Get

To inform the read path and snapshot mechanics driving our `Scan` implementation, we traced a point lookup operation through LevelDB's multi-layered architecture.

4.1 Entry Point and Snapshot Isolation - `DBImpl::Get`

A caller invokes `db->Get(read_options, key, &value)`. `DBImpl::Get()` acquires the mutex and establishes the read horizon. If `read_options.snapshot != nullptr`, the explicitly provided snapshot sequence number is utilized. Otherwise, `versions_->LastSequence()` is captured, acting as an implicit snapshot at the exact moment of the call. This sequence number is subsequently packed into a `LookupKey(key, snapshot_seq)`, encoding the user key alongside the maximum sequence number visible to the reader.

```
MutexLock l(&mutex_);
// Pause reads while ForceFullCompaction is running.
// The database should be briefly unavailable for foreground work
   during
```

```

// manual full compaction.
while (force_compaction_in_progress_) {
    force_compaction_done_.Wait();
}
SequenceNumber snapshot;
if (options.snapshot != nullptr) {
    snapshot =
        static_cast<const SnapshotImpl*>(options.snapshot)->
            sequence_number();
} else {
    snapshot = versions_->LastSequence();
}
LookupKey lkey(key, snapshot);

```

References to the active memtable, immutable memtable, and current Version are secured under the mutex and reference-counted. The mutex is then released for the physical lookup, ensuring reads do not block concurrent write operations.

4.2 Layer 1 - Active MemTable

`mem->Get(lkey, &value, &s)` queries `MemTable::Get()`. This positions a `SkipList` iterator at the first entry $\geq$ `lkey.memtable_key()` via `iter.Seek(memkey.data())`. If the target entry matches the user key, the 8-byte tag is evaluated: `kTypeValue` extracts the value, while `kTypeDeletion` returns `NotFound`. The sequence number embedded within the tag is implicitly filtered by the `Seek` comparator, guaranteeing only versions $\leq$ the snapshot sequence number are matched.

4.3 Layer 2 - Immutable MemTable

If the active memtable yields a miss, the query proceeds to `imm_` (the memtable currently flushing to disk). Because `imm_` strictly predates `mem_`, querying it sequentially preserves correctness.

4.4 Layer 3 - SSTables: `Version::Get`

If both memtables yield misses, `current->Get(options, lkey, &value, &stats)` is invoked. `Version::Get()` delegates the search to `ForEachOverlapping()` utilizing a `State::Match` callback.

4.5 Level-0: Linear Scan Newest-to-Oldest

Because Level-0 SSTables originate directly from memtable flushes, their key ranges may overlap. `ForEachOverlapping()` aggregates every L0 file whose `[smallest, largest]` bracket bounds the user key. It sorts these files newest-first (`NewestFirst` comparator) and probes them iteratively. The first file yielding `kFound` or `kDeleted` terminates the search.

4.6 Levels 1-6: Binary Search

For levels ≥ 1 , SSTables possess strictly non-overlapping key ranges. `ForEachOverlapping()` executes `FindFile(vset->icmp_, files_[level], internal_key)` to binary-search

for the target file. If the identified file bounds the user key, exactly one file is probed per level, reducing lookup complexity to $O(\log(\text{files_per_level}))$ rather than linear scale.

4.7 Table Cache Lookup - `TableCache::Get`

For each candidate file, the `State::Match` callback executes `table_cache->Get()`. `TableCache` maintains an LRU cache of instantiated `Table` objects (parsed SSTable index blocks). If uncached, the file is opened, the index block is parsed, and `Table::InternalGet()` is executed: the index block targets the data block, the bloom filter is verified, and the data block is binary-searched for the key. The `SaveValue` callback ultimately writes the located value into the result string.

4.8 Iterator Construction for Range Reads

For range operations, `DBImpl::NewInternalIterator()` constructs a unified, sorted view across all layers:

```
std::vector<Iterator*> list;
list.push_back(mem_->NewIterator());
mem_->Ref();
if (imm_ != nullptr) {
    list.push_back(imm_->NewIterator());
    imm_->Ref();
}
versions_->current()->AddIterators(options, &list);
Iterator* internal_iter =
    NewMergingIterator(&internal_comparator_, &list[0], list.size());
```

`Version::AddIterators()` appends one iterator per L0 file and one `LevelFileNumIterator`-backed concatenating iterator per level ≥ 1 . `NewMergingIterator()` implements a heap-based k-way merge, ensuring the merged iterator yields the globally next key. Finally, `NewDBIterator()` wraps this structure to surface exclusively user keys, gracefully resolving older versions and deletion markers.

5 Implementing `Scan(start_key, end_key)`

5.1 Design: `NewIterator()` vs. Internal Modification

An alternative approach to implementing `Scan` would involve manually iterating each level's file list, opening individual SSTables, and systematically merging the results. We determined that this approach would redundantly duplicate the merging, snapshot-filtering, and deletion-hiding logic natively provided by LevelDB's iterator API.

Consequently, our `Scan()` implementation directly invokes `NewIterator(read_options)`. This delegates the construction of the full `MergingIterator` stack, providing a unified, snapshot-consistent, and sorted view of the entire database without modifying internal table structures.

5.2 Unified View via `MergingIterator`

`NewInternalIterator()` aggregates child iterators representing the active memtable, immutable memtable, L0 SSTables, and upper-level concatenating iterators. `NewMergingIterator()`

encapsulates these within a min-heap: upon each `Next()` call, it efficiently extracts the child possessing the globally smallest key. `NewDBIterator()` subsequently filters the internal-key stream. The result mirrors the consistent, version-resolved output of a point `Get()` natively executed across all levels.

5.3 Implementation Details

```
Status DBImpl::Scan(const ReadOptions& options, const Slice& start_key,
                  const Slice& end_key,
                  std::vector<std::pair<std::string, std::string>>&
                  result) {
    if (result == nullptr) {
        return Status::InvalidArgument("Result vector can't be null");
    }
    result->clear();

    ReadOptions read_options = options;
    const Snapshot* owned_snapshot = nullptr;
    if (read_options.snapshot == nullptr) {
        owned_snapshot = GetSnapshot();
        read_options.snapshot = owned_snapshot;
    }

    std::unique_ptr<Iterator> it(NewIterator(read_options));
    it->Seek(start_key);

    while (it->Valid() && it->key().compare(end_key) < 0) {
        result->emplace_back(std::string(it->key().data(), it->key().size()),
                           std::string(it->value().data(), it->value().size()));
        it->Next();
    }

    Status s = it->status();
    if (owned_snapshot != nullptr) {
        ReleaseSnapshot(owned_snapshot);
    }
    return s;
}
```

5.4 Interval Bounds Handling

We utilize `it->Seek(start_key)` to position the `MergingIterator` at the first entry $\geq$ `start_key`, effectively fulfilling the inclusive `[` boundary requirement. The loop condition `it->key().compare(end_key) < 0` applies a strict mathematical less-than comparison, ensuring `end_key` is systematically excluded to fulfill the open `)` boundary. If `start_key` $\geq$ `end_key`, the loop correctly evaluates to false immediately, yielding an empty result.

5.5 Snapshot Correctness

Range scans are not atomic at the OS level and may iterate over thousands of entries. Without snapshot isolation, keys inserted by concurrent writes mid-scan could intermit-

tently appear, yielding non-serializable outputs. We address this by implicitly provisioning a private snapshot if the caller omits one. We explicitly preserve caller-supplied snapshots to prevent silently violating a caller’s intended read horizon in complex transaction workflows.

6 Implementing DeleteRange(start_key, end_key)

6.1 Design: Discovery and Atomic Deletion

Because LevelDB natively lacks a range tombstone data format, we architected our solution utilizing two robust primitives: an `Iterator` for key discovery, and a `WriteBatch` for atomic commitment. Our iterator systematically walks the `[start_key, end_key)` boundary, collecting live keys into an internal `WriteBatch` of distinct point deletes via `batch.Delete(key)`. The batch is then committed inline through the standard WAL and memtable pathways, preserving existing log records, sequence numbering, and compaction semantics.

6.2 The TOCTOU Race Condition

A naive iteration-to-batch implementation introduces a critical Time-Of-Check to Time-Of-Use (TOCTOU) race condition. Consider the following timeline:

- **T1:** `DeleteRange` scans the range, capturing existing keys.
- **T2:** A concurrent thread executes `Put("new_key", "val")` within the range boundary, committing with Sequence Number 42.
- **T3:** `DeleteRange` commits its batch. The tombstones are assigned Sequence Number 43.

Because sequence numbers are acquired at the moment of commitment rather than enqueue time, the tombstone (Sequence 43) mathematically overshadows the concurrent write (Sequence 42). The newly inserted key is improperly deleted despite arriving after `DeleteRange` initiated.

6.3 Achieving Linearizability via an Exclusive Writer

To resolve the TOCTOU vulnerability, our implementation secures the leading position within the `writers_` queue throughout the entirety of the scan-and-commit sequence. This effectively blocks concurrent writers from acquiring sequence numbers. We implemented this by leveraging the `exclusive` field flag within the internal `Writer` struct.

```
if (w->exclusive) {  
    // Do not absorb a DeleteRange sentinel into another writer's batch  
    group.  
    break;  
}
```

Our `DeleteRange()` function enqueues itself as an exclusive writer. Once it reaches the head of `writers_`, it intentionally *unlocks* the database mutex. This is a critical engineering requirement: maintaining the mutex during `NewIterator()` triggers an immediate deadlock, as `NewInternalIterator()` independently attempts to lock `mutex_`.

With the mutex safely released, concurrent read operations (`Get()`) proceed unhindered. However, write operations cannot progress because our exclusive sentinel occupies the `writers_.front()` position. Concurrent `Put()` requests append to the queue and deterministically wait on `cv.Wait()`.

```
Status DBImpl::DeleteRange(const WriteOptions& write_options,
                           const Slice& start_key, const Slice& end_key
                           ) {
    WriteBatch batch;
    Writer w(&mutex_);
    w.batch = &batch;
    w.sync = write_options.sync;
    w.done = false;
    w.exclusive = true;

    MutexLock l(&mutex_);
    writers_.push_back(&w);
    while (!w.done && &w != writers_.front()) {
        w.cv.Wait();
    }
    if (w.done) return w.status;

    // We're at the front. Release the mutex so NewIterator can grab it
    // briefly,
    // but new writers still can't run.
    mutex_.Unlock();

    std::unique_ptr<Iterator> it(NewIterator(ReadOptions()));
    it->Seek(start_key);
    while (it->Valid() && it->key().compare(end_key) < 0) {
        batch.Delete(it->key());
        it->Next();
    }
    Status s = it->status();
    it.reset();

    mutex_.Lock();
    if (s.ok()) {
        s = MakeRoomForWrite(false);
    }

    if (s.ok() && WriteBatchInternal::Count(&batch) > 0) {
        uint64_t last_sequence = versions_->LastSequence();
        WriteBatchInternal::SetSequence(&batch, last_sequence + 1);
        last_sequence += WriteBatchInternal::Count(&batch);
        {
            mutex_.Unlock();
            s = log_->AddRecord(WriteBatchInternal::Contents(&batch));
            bool sync_error = false;
            if (s.ok() && write_options.sync) {
                s = logfile_->Sync();
                if (!s.ok()) sync_error = true;
            }
            if (s.ok()) {
                s = WriteBatchInternal::InsertInto(&batch, mem_);
            }
            mutex_.Lock();
            if (sync_error) { RecordBackgroundError(s); }
        }
    }
}
```

```

    }
    versions_ ->SetLastSequence(last_sequence);
}

writers_.pop_front();
if (!writers_.empty()) {
    writers_.front()->cv.Signal();
}
return s;
}

```

Rather than handing the constructed batch off to a secondary `Write()` call, our logic expertly processes the WAL appending and Memtable insertion inline while securely maintaining sequence ownership. This guarantees a linearizable deletion. Any `Put()` arriving after `DeleteRange()` initiates cannot commit until `DeleteRange()` completes. The `Put()` therefore secures a strictly higher sequence number, ensuring its survival.

6.4 Key Design Trade-Offs

- **Queue Isolation over Mutex Locking:** As detailed above, we prioritized the exclusive writer pattern over a prolonged mutex hold to maintain system serializability without generating I/O-bound deadlocks.
- **Point Deletes over Range Tombstones:** Implementing native range tombstones (analogous to RocksDB) necessitates new SSTable key types, compaction merge loop modifications, and WAL format adjustments. Our point-delete methodology achieved the target semantics via a minimal code footprint, cleanly delegating space reclamation to existing `DoCompactionWork()` pathways.
- **Empty-Range Optimization:** If no keys populate the target range, the formulated batch remains empty. We dynamically bypass WAL and memtable insertion overhead via `WriteBatchInternal::Count(&batch) > 0`, optimizing empty-range calls into $O(1)$ operations.

7 Implementing ForceFullCompaction()

7.1 FullCompactionStats Data Structure

To accurately collect and isolate the performance statistics mandated for manual compactions, we introduced the `FullCompactionStats` struct within `db/db_impl.h`:

```

struct FullCompactionStats {
    FullCompactionStats()
        : compactions(0), input_files(0), output_files(0),
          bytes_read(0), bytes_written(0) {}
    int64_t compactions;
    int64_t input_files;
    int64_t output_files;
    int64_t bytes_read;
    int64_t bytes_written;
};

```

We declare a pointer, `force_full_compaction_stats_`, referencing the caller's stack-allocated instance. By assigning this pointer prior to executing `CompactRange()` and

nullifying it immediately afterward, we systematically eliminate the risk of delayed background compactions writing to a dangling pointer address.

7.2 Synchronization Barrier and Concurrency Control

To robustly serialize concurrent manual compactions, we integrated a conditional synchronization barrier into DBImpl:

```
bool force_compaction_in_progress_ GUARDED_BY(mutex_);
port::CondVar force_compaction_done_ GUARDED_BY(mutex_);
```

Upon entry to ForceFullCompaction(), the executing thread acquires `mutex_` and evaluates a `while (force_compaction_in_progress_)` loop. Concurrent callers efficiently sleep via `force_compaction_done_.Wait()`. Upon exiting the loop, the thread asserts `force_compaction_in_progress_ = true` and mounts the statistics pointer. Upon completion of `CompactRange()`, the thread reacquires the mutex, dismounts the pointer, resets the operational flag, and executes `SignalAll()` to awaken queued callers. This identical flag is evaluated within `Get()` and external `Write()` paths, ensuring the database correctly stalls all foreground operations during manual compaction execution.

7.3 Triggering and Tracking the Compaction

We execute the physical merge operation by invoking LevelDB's native `CompactRange(nullptr, nullptr)` routine. To capture the resulting statistics, we integrated direct tracking hooks into `WriteLevel0Table` and `DoCompactionWork`:

```
if (force_full_compaction_stats_ != nullptr) {
    force_full_compaction_stats_->compactions++;
    force_full_compaction_stats_->output_files += (meta.file_size > 0) ?
        1 : 0;
    force_full_compaction_stats_->bytes_written += meta.file_size;
}

// And within DoCompactionWork():
if (force_full_compaction_stats_ != nullptr) {
    force_full_compaction_stats_->compactions++;
    force_full_compaction_stats_->input_files +=
        compact->compaction->num_input_files(0) +
        compact->compaction->num_input_files(1);
    force_full_compaction_stats_->output_files += compact->outputs.size();
    ;
    force_full_compaction_stats_->bytes_read += stats.bytes_read;
    force_full_compaction_stats_->bytes_written += stats.bytes_written;
}
```

7.4 Printing to stdout

Following the compaction, the recorded variables are actively flushed to `stdout` using casting conversions that ensure proper sizing compatibility:

```
std::fprintf(
    stdout,
    "ForceFullCompaction statistics:\n"
    "  compactions executed: %llu\n"
    "  input files: %llu\n"
```

```

    "output_files: %llu\n"
    "total_bytes_read: %llu\n"
    "total_bytes_written: %llu\n",
    static_cast<unsigned long long>(full_compaction_stats.compactions),
    static_cast<unsigned long long>(full_compaction_stats.input_files),
    static_cast<unsigned long long>(full_compaction_stats.output_files)
    ,
    static_cast<unsigned long long>(full_compaction_stats.bytes_read),
    static_cast<unsigned long long>(full_compaction_stats.bytes_written
    ));
std::fflush(stdout);

```

7.5 Design Trade-Off: Disk I/O vs. Memory Usage

During early iterations, we considered integrating `ApproximateMemoryUsage()` to represent the byte output of flushed memtables. We ultimately discarded this approach. Because memtable flushes execute from RAM, labeling them as "bytes read" artificially inflates apparent disk I/O. We restricted our `bytes_read` metric exclusively to the physical byte size of on-disk SSTable inputs.

8 Compaction Mechanism Analysis

8.1 Structural Properties and Invariants

Throughout our implementation, we carefully analyzed the properties preserved by LevelDB compactions. We noted that while Level-0 files may permissibly overlap, layers ≥ 1 rigorously enforce non-overlapping key bounds. Compaction sequences guarantee that overwritten key versions are definitively discarded, systematically executing the physical space reclamation necessitated by our `DeleteRange` tombstones.

8.2 DoCompactionWork and File Splitting

We observed that `DBImpl::DoCompactionWork()` processes merged inputs in strict internal-key sequence. It tracks the current user key and intelligently bypasses older sequence variants once an authoritative recent version has been successfully emitted across all active read horizons. Furthermore, compaction output is dynamically partitioned rather than rendered as a monolithic SSTable. New output files are initiated upon reaching `MaxOutputFileSize()`, or when `ShouldStopBefore()` flags excessive overlap with granular grandparent files.

8.3 Interaction with Automatic Compaction

LevelDB's internal heuristic (`PickCompaction()`) autonomously triggers background activity based upon designated level-size or seek pressures. Our `ForceFullCompaction()` API introduces a formalized manual path leveraging this exact background machinery. We verified that both protocols successfully coexist.

9 Evaluation

9.1 Functional Correctness

Our implementation successfully completed all evaluation protocols, passing 8 out of 8 functional correctness tests. This confirms:

- **Scan** correctly executes half-open mathematical boundaries, maintains snapshot isolation, and safely no-ops on inverted parameters.
- **DeleteRange** successfully masks deleted keys from subsequent evaluations.
- The exclusive writer discipline implemented within **DeleteRange** deterministically handles heavy concurrent queue load, appropriately sequencing interleaved **Put** and delete operations without data leakage.
- **ForceFullCompaction** correctly restricts foreground writes without deadlocking the background thread pool, maintaining database integrity throughout concurrent API solicitations.

9.2 Performance and Latency Considerations

Our engineering choices prioritize strict functional correctness over theoretical maximal throughput. **DeleteRange** executes in $O(k)$ time relative to the constituent keys. Because the exclusive writer locks the `writers_` queue for the duration of the scan and commit phases, concurrent writes stall. While this presents latency implications for excessively massive key ranges, this operational profile was deemed necessary to enforce TOCTOU-proof linearizability without destructively modifying foundational WAL or SSTable parsing logic.

9.3 Compaction Statistics Accuracy

While our implementation tracks all manual executions precisely, we evaluated an architectural edge case concerning concurrent system operations. The integrated **ForceFullCompaction** statistical hooks capture deltas restricted to the explicit `DoCompactionWork` pathway. Consequently, if LevelDB autonomously executes concurrent auto-compactions (frequently executed as algorithmic "Trivial Moves" bypassing the main work loop), those specific systemic operations are omitted from the manual statistics tally. We concluded this was an acceptable operational trade-off to ensure a strictly non-invasive statistical patch that protects against background thread deadlocks.

9.4 Sample Compaction Statistics

To validate the correctness of our manual compaction pathway, we executed **ForceFullCompaction()** on a populated multi-level database and observed the statistics emitted upon synchronous completion. A representative terminal output is shown below:

```
ForceFullCompaction statistics:
  compactions executed: 4
  input files: 11
  output files: 6
```

```
total bytes read: 8457216
total bytes written: 6234981
```

The reported values indicate that the system successfully traversed multiple LSM-tree levels and invoked the native merge pipeline repeatedly until no further full-compaction work remained. The ratio between bytes read and bytes written additionally reflects removal of overwritten versions, tombstones, and redundant historical entries during merge execution.

- **Compactions Executed:** Total internal compaction jobs triggered during the full compaction sequence.
- **Input Files:** Aggregate number of SSTables selected as merge sources.
- **Output Files:** Aggregate number of newly generated SSTables.
- **Total Bytes Read:** Physical on-disk bytes consumed from source files.
- **Total Bytes Written:** Bytes emitted into newly materialized SSTables.

9.5 Testing Methodology

To rigorously validate semantic correctness, concurrency safety, and compaction behavior, we evaluated the implementation using a diverse collection of deterministic workloads and edge-case scenarios.

- **Range Scan Validation:** Inserted sequential and sparse keys, then verified that `Scan(start_key, end_key)` returned exactly the half-open interval `[start_key, end_key)` in sorted order with latest visible values.
- **Snapshot Consistency Checks:** Executed concurrent writes during long-running scans and confirmed that private or caller-supplied snapshots preserved a stable read horizon.
- **Range Delete Validation:** Populated overlapping key intervals, invoked `DeleteRange()`, and verified that affected keys became logically invisible while keys outside the interval remained intact.
- **Concurrency Stress Testing:** Interleaved multiple `Put()`, `Delete()`, and `DeleteRange()` requests under contention to confirm that the exclusive writer discipline prevented TOCTOU anomalies.
- **Manual Compaction Testing:** Constructed databases spanning multiple levels, invoked `ForceFullCompaction()`, and verified synchronous completion together with correctly formatted statistics output.
- **Boundary Conditions:** Tested empty ranges, inverted bounds, missing keys, repeated deletions, and already-compacted databases to ensure graceful no-op behavior.

All observed outputs were cross-checked against the expected logical database state derived from LevelDB sequence-order semantics.

10 Conclusion

Our submitted implementation successfully augments the LevelDB API while rigorously adhering to its foundational architectural constraints. `Scan()` provides a durable wrapper interfacing with the established `MergingIterator` stack. `DeleteRange()` resolves interval boundaries via safe, sequence-ordered point tombstones governed by an exclusive writer paradigm. `ForceFullCompaction()` formally invokes the manual compaction pipeline while surfacing required operational metrics. By systematically relying upon LevelDB’s standard WAL, memtable, snapshot, and compaction mechanisms, our extensions fulfill the requisite semantic targets securely and elegantly.